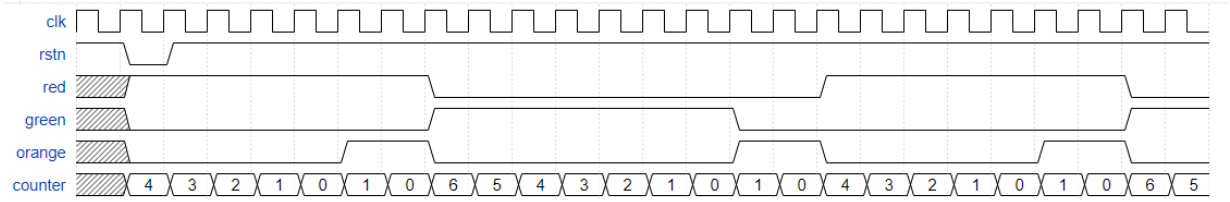
Lab 4: Sequential circuits and FSM

1. a. Implement a state machine to control a traffic light. The FSM should implement the states Green, Red, Orange, Red&Orange, and transitions with the corresponding delays, which will be specified as a parameter. Below, you can find a sample of a timing diagram of its operation.



Module parameters

Name	Width	Description
RED_DELAY	--	Duration of Red state
RED_ORANGE_DELAY	--	Duration of RedOrange state
GREEN_DELAY	--	Duration of Green state
ORANGE_DELAY	--	Duration of Orange state
COUNTER_WIDTH	--	Bit-width of the counter

Module pinout

Name	Direction	Width	Description
clk	in	1	Clock signal
rstn	in	1	Active low reset
red	out	1	Signals when the red light should be on
orange	out	1	Signals when the orange light should be on
green	out	1	Signals when the green light should be on

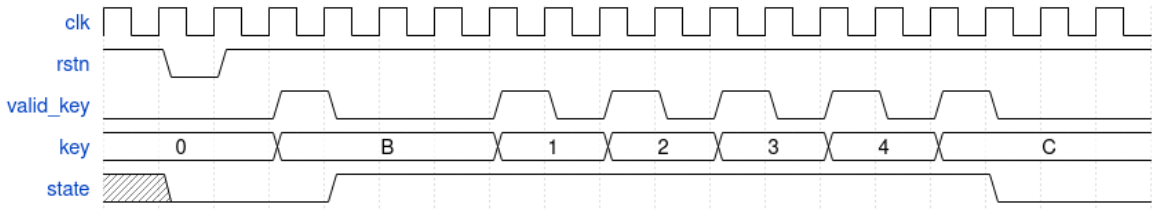
b. Implement the traffic light on FPGA.

For this task, you need to connect your output signals to the RGB IO of the FPGA. You can use the blue light instead of the orange light.

Hint: Your traffic light module should work with a slower clock than the FPGA clock. See https://github.com/silagokth/il2234-lab-demo.

Bonus: Try to show orange light with RGB.

2. a. Implement a state machine to control a 4-digit digital lock. The lock has a keyboard with digits 0-9 and letters A-F. (It can be interpreted as hexadecimal input). Below, you can see an example of locking and unlocking the code using 1234 as a pin.



After reset, the lock starts in the unlocked state. After that, the user can lock it by pressing key B. Once in the locked state, the lock can be unlocked by pressing the 4 correct digits followed by key C. The user can change the password while in the unlocked state by pressing the key E followed by the 4 digits of the new password.

Note that the signal valid_key should be present in all transitions, but it has been omitted to simplify the diagram.

Module pinout

Name	**Direction**	**Width**	**Description**
clk	in	1	Clock signal
rstn	in	1	Active low reset
key	in	4	The "value" of the pressed key
valid_key	in	1	1 when the user presses a key
state	out	1	1: locked, 0: unlocked

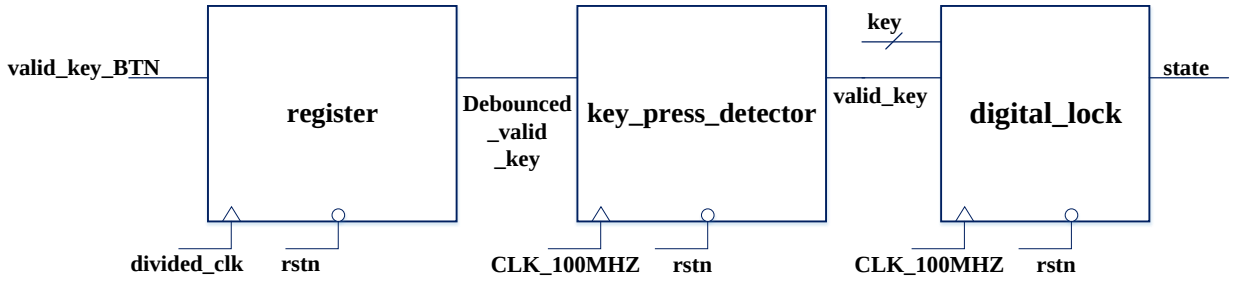
b. Implementing the digital lock system on FPGA.

For this task, the lock's state can be mapped to an LED on the FPGA board, indicating whether it is locked or unlocked. You can use the switches on the FPGA board to simulate the keypad inputs (key) and use the push button for the valid_key signal. When using mechanical switches or buttons as inputs on the FPGA, it's essential to account for **debouncing**. Mechanical switches can create noise and multiple transitions (bounces) when pressed or released, which may cause the FPGA to register multiple unintended inputs. The valid_key input signal connected to the button should be debounced by passing it through a register. The register will sample the input at a much lower frequency (100 Hz), which helps to stabilize the signal by allowing it to settle before being processed.

Another challenge of implementing this module on FPGA is detecting when the Switch/button is active and capturing the key's correct digit. For this, you need to implement an FSM that can detect when the Switch/button is activated(press). This FSM must have two states: **Released and**

Pressed.

Released State: When the key is not being pressed, the FSM remains in this state.

Pressed State: When the key is pressed (input signal goes high), the FSM transitions from Released to Pressed. The FSM's output becomes active when this transition occurs, indicating a valid key press. The output should be asserter only for one clock cycle.

The FSM should register its output on the positive edge of the clock signal (clk). See the figure below to understand how you should connect the modules.